Campus Mobility Platform

Design Report for Real-Time Campus Ride Management

Cult Open Projects 2026 - Software Development

12 June 2026

1. Problem Understanding

IIT Roorkee campus transport depends on quick coordination between passengers and e-rickshaw drivers. In an informal workflow, passengers do not know which drivers are available, drivers do not have a central request queue, and coordinators have limited visibility into demand, completed trips, and driver performance.

The Campus Mobility Platform is a responsive web MVP that centralizes this workflow for three roles:

- **Passenger:** create a profile, request a ride, track live ride status, and rate a completed ride.
- **Driver:** manage online/off-line availability, view incoming requests, accept or reject a ride, and move the ride through its lifecycle.
- **Admin/Coordinator:** monitor active rides, available drivers, completed trips, demand hotspots, ratings, and ride history.

The project focuses on the core engineering component required by the problem statement: realtime ride-state synchronization. It uses Server-Sent Events so that every browser connected to the app receives the latest ride, driver, and analytics state without manual refresh.

Scope Implemented

- Demo-style registration/login through role-specific profile creation and selection.
- Driver onboarding fields such as name, vehicle number, location, and availability.
- Ride request, assignment, status tracking, completion, and rating workflows.
- Realtime updates through a live event stream.
- Operations dashboard with active rides, fleet availability, demand charts, leaderboard, feedback, and recent history.

Out of Scope for MVP

Live maps, UPI payments, ride scheduling, and ML-based demand forecasting are future extensions. They are intentionally excluded to keep the implementation reliable, reproducible, and complete within the project deadline.

2. System Architecture

The system uses a simple three-layer architecture that can run locally without external services.

```
Browser UI
Passenger Tab | Driver Tab | Admin Tab
        |
        | HTTP JSON APIs + Server-Sent Events
        v
Node.js HTTP Server
Static File Server | API Router | SSE Broadcaster | Analytics Builder
        |
        | File read/write
        v
JSON Persistence
data/store.json
```

Technology Stack

- **Frontend:** HTML, CSS, and vanilla JavaScript.
- **Backend:** Node.js built-in `http`, `fs`, `path`, and `url` modules.
- **Realtime:** Server-Sent Events at `GET /api/events`.
- **Persistence:** JSON file store at `data/store.json`.
- **Dependencies:** none beyond Node.js.

Realtime Flow

1. Browser loads the app and calls `GET /api/state`.
2. Browser opens `GET /api/events` and stays subscribed.
3. A passenger, driver, or admin action calls a JSON API.
4. The server validates the action, updates `data/store.json`, recomputes analytics, and broadcasts the new state.
5. All connected dashboards re-render from the same authoritative state.

Deployment and Reproducibility

The app runs with:

```
npm start
```

The app then opens at:

```
http://localhost:3000
```

No database setup, API key, package installation, or external network service is required. This makes the repository easy for evaluators to clone, run, inspect, and test.

3. Database Schema and ERD

For the MVP, the database is represented by a JSON document. The schema is intentionally close to a relational model, so it can later be migrated to PostgreSQL, MySQL, or MongoDB.

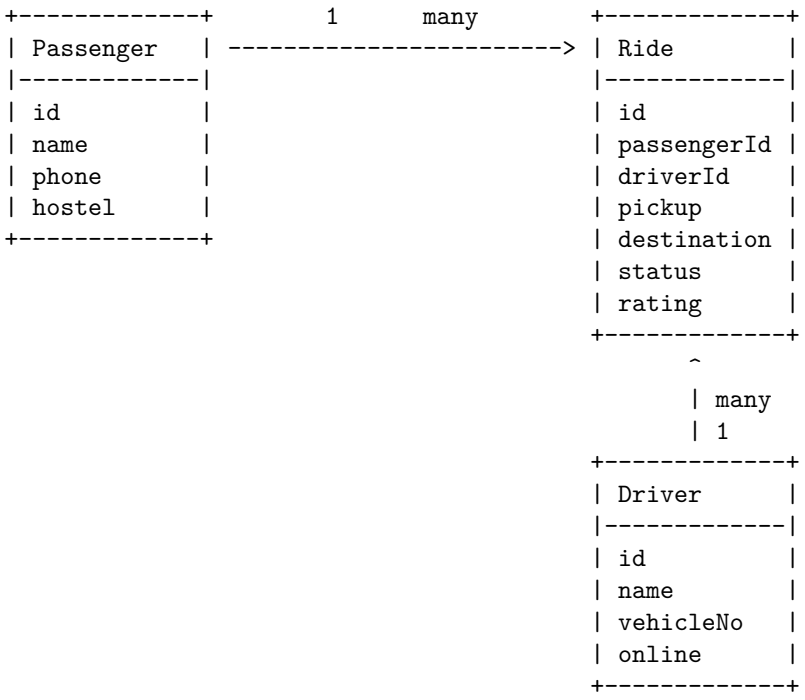
Entities

Entity	Key Fields	Purpose
Passenger	`id`, `name`, `phone`, `hostel`	Stores passenger identity and demo login/profile data.
Driver	`id`, `name`, `phone`, `vehicleNumber`, `vehicleType`, `currentLocation`, `online`, `verified`	Stores driver onboarding, availability, and fleet data.
Ride	`id`, `passengerId`, `driverId`, `pickup`, `destination`, `seats`, `notes`, `status`, timestamps	Stores every ride request and lifecycle state.
Rating	`score`, `comment`, `createdAt`	Stores passenger feedback for completed rides.

Ride Status Values

```
requested -> accepted -> in_progress -> completed
requested -> cancelled
accepted  -> cancelled
```

Entity Relationship Diagram



Data Integrity Rules

- A passenger can have only one active ride at a time.
- A ride can be assigned to only one driver.
- Only online drivers can accept ride requests.
- A driver can hold only one active assignment.
- Only the assigned driver can start or complete a ride.
- Ratings are accepted only after ride completion.

4. API Overview

The backend exposes JSON APIs for state, profiles, ride workflow, ratings, and analytics. Every write operation persists data and broadcasts the updated state to connected clients.

Method	Endpoint	Purpose
GET	`/api/health`	Confirms the server is running.
GET	`/api/state`	Returns passengers, drivers, rides, and analytics.
GET	`/api/events`	Opens the realtime Server-Sent Events stream.
GET	`/api/analytics`	Returns the analytics snapshot.
POST	`/api/login`	Creates or returns a demo passenger or driver profile.
POST	`/api/rides`	Creates a new passenger ride request.
POST	`/api/rides/:id/accept`	Assigns an online driver to a requested ride.
POST	`/api/rides/:id/reject`	Records that a driver rejected a request.
POST	`/api/rides/:id/status`	Moves a ride through valid lifecycle states.
POST	`/api/rides/:id/rating`	Stores passenger rating and feedback.
POST	`/api/drivers/:id/availability`	Updates driver online/off-line status and location.
POST	`/api/reset`	Restores seeded demo data.

Backend Logic

The server is deliberately implemented with Node.js built-ins. This keeps the project transparent and easy to evaluate. The API router parses JSON bodies, validates required fields, rejects invalid state transitions, writes the updated store, and sends the updated public state to every SSE client.

Analytics Computed

- Total rides, active rides, completed rides, cancelled rides.
- Online drivers and available drivers.
- Average rating across rated rides.
- Popular pickup points and destinations.
- Driver performance: completed rides, active ride, online status, and average rating.

These values are recalculated from the store on every state response, so the dashboard remains consistent with the persisted ride data.

5. User Workflows and Verification

Passenger Workflow

1. Select an existing passenger or create a new passenger profile.
2. Enter pickup, destination, seat count, and optional notes.
3. Submit a ride request.
4. Watch the ride status update live: requested, accepted, in progress, completed.
5. Rate the ride after completion.

Driver Workflow

1. Select an existing driver or create a new driver profile.
2. Go online and update current location.
3. View pending ride requests.
4. Accept or reject a request.
5. Start the accepted ride and then complete it.

Admin Workflow

The Admin dashboard shows:

- Active ride table.
- Driver availability table.
- Popular pickup points.
- Popular destinations.
- Driver leaderboard.
- Passenger feedback.
- Recent ride history.

Verified End-to-End Flow

The implemented system was tested through the complete demo path:

```
Passenger request
-> Driver accept
-> Driver start ride
-> Driver complete ride
-> Passenger rating
-> Admin analytics update
```

Additional automated smoke coverage verifies health checks, SSE initial state, duplicate active ride prevention, off-line driver rejection, single-driver assignment, lifecycle transitions, ratings, analytics, and demo reset.

Run checks with:

```
npm run check
npm run smoke
```

6. Design Decisions and Future Improvements

Design Decisions

- **Server-Sent Events over WebSockets:** The app needs reliable server-to-browser state broadcasts. SSE is simpler, dependency-free, and enough for this MVP.
- **JSON persistence:** A file store keeps the project reproducible without requiring a database service. The model still maps cleanly to relational tables.
- **Role tabs in one page:** Passenger, Driver, and Admin workflows are visible in one app, which makes the demo fast and easy to evaluate.
- **Demo profile creation instead of password auth:** The project demonstrates registration/login behavior through profile creation and selection while prioritizing the required realtime ride workflow.
- **Conservative UI:** The dashboard uses compact cards, tables, status badges, and responsive grids so it feels like an operational tool rather than a marketing page.

Known Limitations

- No live map or geolocation stream.
- No payment flow.
- No scheduled rides.
- No password hashing or production session management.
- JSON file persistence is not intended for concurrent production-scale production use.

Future Improvements

- Add PostgreSQL with normalized passenger, driver, ride, and rating tables.
- Add secure sessions or JWT authentication.
- Add live maps with OpenStreetMap or Google Maps.
- Add ride scheduling and QR/UPI payment simulation.
- Add audit logs for dispatch and lifecycle events.
- Add demand forecasting using historical ride timestamps and pickup clusters.

Submission Readiness

The repository includes source code, configuration, documentation, a reproducible run command, a smoke test, and this design report. The remaining external deliverable is the maximum three-minute demonstration video showing registration/profile selection, ride request, assignment, realtime updates, driver dashboard, and ratings.